



RAPPORT DE PROJET 3A

Antonin LARVOR

PROMOTION 2025

27 FÉVRIER 2025

Création d'un chatbot pour le site de Seatech

Tuteur de Projet : Farid DINAR



Sommaire

1	Introduction	2
1.1	Contexte et Motivation	2
2	Introduction au Traitement du Langage Naturel (NLP)	3
2.1	Embeddings et Représentation Vectorielle	3
2.2	Transformers et Modèles de Langage	3
3	Développement d'un Système de RAG pour le Chatbot	4
4	Technologies Utilisées	6
4.1	API de Firecrawl	6
4.2	Backend en Python	6
4.3	Site Web	6
4.4	Serveur avec Flask	6
4.5	Modèles d'IA et Embeddings	6
5	Conclusion	8

1 Introduction

1.1 Contexte et Motivation

Les systèmes de RAG (Retrieval-Augmented Generation) représentent une avancée significative dans le domaine de l'intelligence artificielle. Ils combinent les capacités des grands modèles de langage (LLM) avec des bases de connaissances externes pour générer des réponses plus précises et contextuellement appropriées. Cette technologie permet d'intégrer de nouvelles informations en continu, rendant les assistants virtuels plus fiables et adaptables.

L'objectif principal de ce projet était de créer un chatbot capable de répondre à des questions relatives à l'école d'ingénieurs Seatech en utilisant des informations provenant majoritairement du site internet de l'école.

J'ai choisi ce projet car j'ai déjà travaillé sur un système de RAG pendant 4 mois cet été lors de mon stage de deuxième année. J'ai repris une grande partie de l'architecture car j'avais adoré travailler dessus et il me restait plein de points à approfondir. De plus, mon stage de fin d'étude risque d'utiliser des technologies de RAG ou du moins des LLM. Ce projet était donc une opportunité pour me perfectionner dans ce domaine.

2 Introduction au Traitement du Langage Naturel (NLP)

Le NLP (Natural Language Processing) est une branche de l'intelligence artificielle qui vise à permettre aux machines de comprendre et générer du langage humain. Les applications incluent les chatbots, la traduction automatique et les moteurs de recherche avancés. Notre chatbot s'appuie sur ces technologies pour interagir efficacement avec les utilisateurs.

2.1 Embeddings et Représentation Vectorielle

Les embeddings sont essentiels en NLP, permettant de transformer des mots ou phrases en vecteurs numériques. Ces représentations facilitent la comparaison sémantique entre différents éléments du langage. C'est en ce point que les embeddings contribuent au système de RAG, en faisant de la recherche avancée entre la question de l'utilisateur et les textes contenus dans la base de données.

2.2 Transformers et Modèles de Langage

Les LLMs (Large Language Models) sont des modèles d'intelligence artificielle capables de comprendre et générer du texte en langage naturel. Ils utilisent une architecture appelée Transformer, qui leur permet de traiter le contexte d'un texte efficacement grâce à un mécanisme d'attention.

Cependant, ces modèles ont une limite majeure : ils ne peuvent pas accéder à des informations mises à jour après leur entraînement. C'est là qu'intervient le RAG (Retrieval-Augmented Generation).

Le RAG améliore les réponses des LLMs en leur permettant d'accéder à des bases de données ou à des documents externes en temps réel. Au lieu de se baser uniquement sur leur entraînement, ils récupèrent des informations pertinentes avant de générer une réponse.

3 Développement d'un Système de RAG pour le Chatbot

Un chatbot basé sur le Retrieval-Augmented Generation (RAG) combine récupération d'information et génération de texte. Contrairement aux chatbots classiques, il recherche d'abord les documents pertinents avant de générer une réponse adaptée.

On peut schématiser la chaîne de traitement (pipeline) d'un système de RAG simple tel qu'on peut le voir sur la figure 1. On peut décomposer cette chaîne en deux, d'un côté la préparation des données, de l'autre le traitement des données avec la recherche et la génération de réponse.

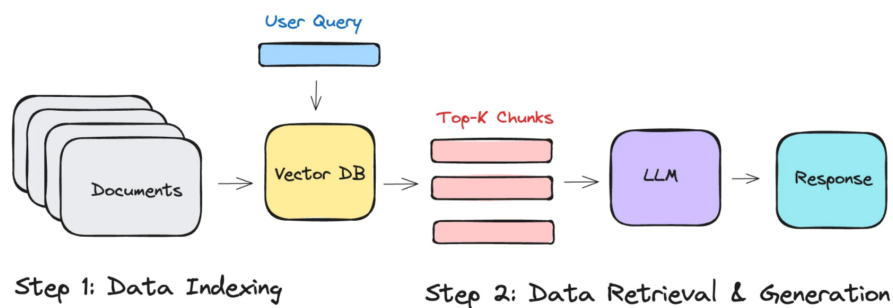


FIGURE 1 – Basic RAG pipeline

- Étape 1 : Préparation des données

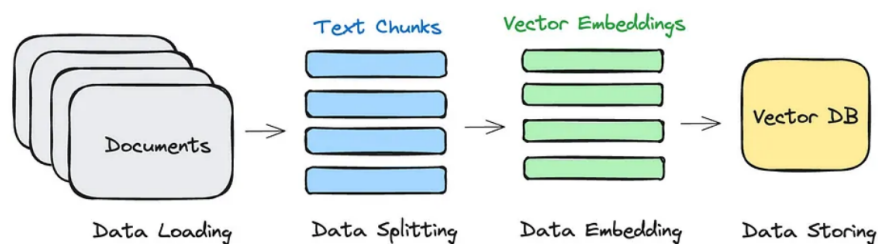


FIGURE 2 – Data Indexing

La première étape, schématisée par la figure 2, consiste à constituer la base de données du système avec des documents textuels. En l'occurrence nos documents textuels sont extraits du site internet de l'école et stocké dans un fichier *JSON*.

Un document unique peut contenir des centaines de pages, des milliers de caractères et couvrir une multitude de sujets différents. Pour que les documents soient lisibles et interprétables par notre système, il faut les diviser en morceaux de texte, que nous appellerons chunks par la suite. Ces chunks sont ensuite représentés par des vecteurs

d'embeddings qui capturent le contexte contenu dans ces chunks et permettent la recherche avancée par la suite. La création de ces chunks est cruciale pour l'efficacité du système, car une fenêtre de contexte trop petite peut manquer d'informations, tandis qu'une fenêtre trop large peut introduire du bruit et perdre la partie la plus importante du contexte.

Ces vecteurs d'embeddings associés aux différents chunks sont ensuite stockés dans une autre base de données, cette fois-ci vectorielle, interne au système.

En résumé, la préparation des données consiste à lire les documents, les séparer en petits morceaux de texte, associer à chaque morceau une représentation vectorielle au sein de l'espace d'embeddings, et stocker cette représentation dans une base de données vectorielle.

- Étape 2 : Recherche et Génération

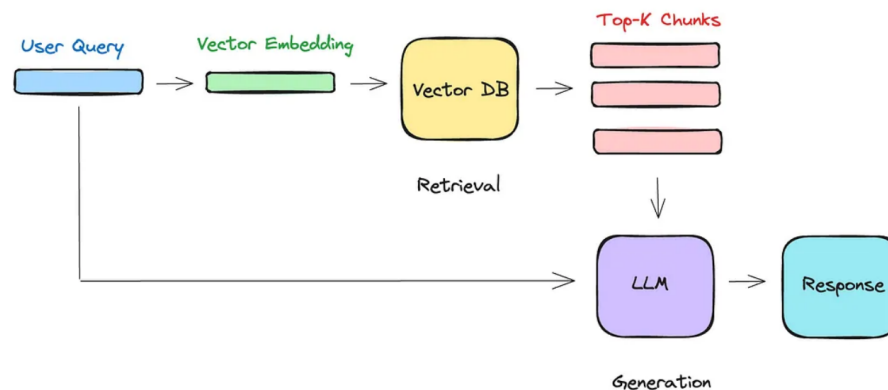


FIGURE 3 – Data Retrieval and Generation

Une fois que la base de données est constituée, l'utilisateur peut effectuer des requêtes, ou queries en anglais, le processus est détaillé figure 3.

Dans un premier temps, la requête de l'utilisateur est lue par le modèle d'embeddings afin de lui attribuer un vecteur dans l'espace d'embeddings. Ce vecteur est comparé aux vecteurs associés aux chunks stockés dans notre base de données, et le système fait ressortir les chunks dont le contexte est le plus proche de la requête de l'utilisateur dans l'espace d'embeddings.

Dans un second temps, la requête de l'utilisateur ainsi que les chunks trouvés lors de la première étape sont donnés au LLM pour générer une réponse à la question de l'utilisateur en se basant uniquement sur le contexte relevé, à l'aide d'un prompt spécifique.

4 Technologies Utilisées

4.1 API de Firecrawl

L'API de Firecrawl a été utilisée pour obtenir la base de données. Cette API combine le webscrapping avec un LLM pour obtenir des pages de texte directement prêtes à l'emploi. Cette fonctionnalité a été cruciale pour constituer une base de données propre et utilisable sans trop d'efforts manuels.

4.2 Backend en Python

Le backend du chatbot a été entièrement codé en Python. Les agents proposés par Langchain ont été majoritairement utilisés pour implémenter les fonctionnalités du chatbot. Langchain fournit des outils puissants pour interagir avec des modèles de langage et des bases de connaissances, facilitant ainsi le développement de systèmes de RAG.

4.3 Site Web

Le site web du chatbot a été créé avec HTML, CSS, et JavaScript natif. Cette approche a permis de développer une interface utilisateur interactive et réactive. Le site web sert de point d'entrée pour les utilisateurs qui souhaitent poser des questions au chatbot.

4.4 Serveur avec Flask

Le serveur du chatbot a été développé avec Flask, un micro-framework pour Python. Flask a été choisi pour sa simplicité et sa flexibilité, permettant de créer rapidement un serveur web fonctionnel. Le serveur tourne localement sur mon PC, facilitant ainsi les tests et le développement.

4.5 Modèles d'IA et Embeddings

Plusieurs modèles d'IA et techniques d'embeddings ont été utilisés dans ce projet :

- **Mistral** : Le modèle privilégié pour ce projet en raison de sa nature open source et de sa compatibilité avec la langue française. Mistral offre une alternative performante et économique, tout en permettant une exécution locale sans dépendance à des API payantes.
- **API de OpenAI** : Utilisée pour tester rapidement l'application avec des modèles de

langage performants. OpenAI propose des modèles propriétaires accessibles via une API payante. Ces modèles sont optimisés pour la performance et la rapidité, mais nécessitent un coût d'utilisation.

- **API de Voyage** : VoyageAI propose des modèles d'embeddings très performant qui ont été privilégié pour le modèle d'embeddings du système. Voyage offre un grand nombre de tokens gratuits, largement suffisant pour notre projet.
- **API de Groq** : Testée pour ses capacités de traitement rapide et efficace. Groq propose une API disposant des meilleurs modèles avec un certain nombre de token gratuits.
- **Ollama** : Utilisé pour faire tourner les modèles localement, réduisant ainsi la dépendance aux API externes. Ollama permet de déployer des modèles open source localement, ce qui peut être plus économique mais potentiellement plus lent en termes de performance.

La différence principale entre les modèles open source et les modèles propriétaires réside dans le coût et la performance. Les modèles open source, comme Mistral, peuvent être exécutés localement à moindre coût, mais ils peuvent être plus lents et nécessiter des ressources matérielles importantes. En revanche, les modèles propriétaires, accessibles via des API payantes comme OpenAI ou Voyage, offrent des performances optimisées et une rapidité accrue, mais à un coût plus élevé. Le choix entre ces deux types de modèles dépend des besoins spécifiques du projet, des contraintes budgétaires, et des ressources disponibles.

5 Conclusion

En conclusion, ce projet a permis de développer un chatbot performant et adaptable grâce à la technologie RAG. Les défis rencontrés, tels que la constitution d'une base de données propre et l'intégration de modèles d'IA, ont été surmontés avec succès. Les retours des utilisateurs ont été très positifs, soulignant la qualité et la pertinence des réponses fournies par le chatbot.

Ce projet a également été une excellente opportunité pour approfondir mes compétences dans le domaine des LLM et des systèmes de RAG. Les technologies utilisées, telles que l'API de Firecrawl, les agents de Langchain, et les modèles d'IA comme Mistral, ont permis de créer un système robuste et efficace.

Pour les perspectives d'amélioration, il serait bénéfique d'enrichir la base de données en impliquant davantage de membres de l'école. Cela permettrait de couvrir un plus large éventail de questions et d'améliorer encore la précision des réponses. De plus, l'optimisation des modèles d'IA et des embeddings pourrait accroître la rapidité et l'efficacité du système.

En somme, ce projet a non seulement atteint ses objectifs initiaux, mais a également ouvert la voie à de futures améliorations et innovations dans le domaine des chatbots basés sur la technologie RAG.